

Generative Modelling for Extremes and Risk

Edinburgh Summer School on Generative AI for Extremes

THE UNIVERSITY *of* EDINBURGH

Lead	Jordan Richards; University of Edinburgh
Dates	8–11 September, 2026
Webpage	https://gamex-network.github.io/school/
Software	Python 3.10+; PyTorch; JupyterLab
Runtime	2 hours

Materials	Three Jupyter notebooks: standard GANs, EV-GAN, and ExceedGAN.
Before the session	Please bring a laptop with Python and the required packages installed; run JupyterLab once to check the environment.
Output	Trained GAN-based generators and diagnostic plots comparing bulk fit, upper-tail quantiles, and bivariate extremal behaviour.

Session Aims

By the end of this tutorial, participants should be able to:

1. Translate the mathematical GAN formulation into a PyTorch generator, discriminator, loss function, and alternating optimisation loop.
2. Explain why a standard ReLU GAN can struggle with unbounded heavy-tailed distributions, and how EV-GAN modifies the generator using extreme-value theory.
3. Construct and assess an ExceedGAN model for multivariate threshold exceedances, and compare it with standard GAN and EV-GAN using tail and dependence diagnostics.

Software Setup

A local JupyterLab installation is the recommended setup. The notebooks automatically use CUDA when available, Apple's MPS backend on compatible Macs, and otherwise fall back to CPU.

```
python3 -m venv gan_ext_env
source gan_ext_env/bin/activate
python -m pip install --upgrade pip
pip install torch numpy matplotlib scipy statsmodels jupyter
jupyter lab
```

On Windows, activate the environment with `gan_ext_env\Scripts\activate` instead of the `source` command above.

Required Packages

Language	Packages	Use
Python	<code>torch</code> , <code>numpy</code> , <code>matplotlib</code>	GAN construction, optimisation, simulation and plotting.
Python	<code>scipy</code> , <code>pandas</code> , <code>statsmodels</code>	Burr margins, TIF correction, Gumbel copula simulation and diagnostic tables.
Notebook	<code>jupyterlab</code>	Running and editing the practical notebooks.

Data & Files

All data used in the practical are simulated inside the notebooks; there is no external data download.

Standard GAN `01_standard_GAN.ipynb` – notation, PyTorch construction, adversarial training, and diagnostics.

EV-GAN `02_evGAN_vs_standard_GAN_1d.ipynb` – standard GAN versus EV-GAN on a one-dimensional heavy-tailed Burr distribution.

ExceedGAN `03_exceedGAN_vs_evGAN_bivariate.ipynb` – bivariate Gumbel–Burr exceedances and comparison of GAN, EV-GAN, and fixed-level ExceedGAN.

Workshop runtime Approximately 2 hours; extension exercises are optional if time is short.

Key Ideas

A standard GAN consists of a generator G_θ and discriminator D_ϕ . For latent input $Z \sim \pi$, the generator produces $\tilde{X} = G_\theta(Z)$ and the discriminator tries to separate $\tilde{X}$ from observed data X . The classical objective is

$$\min_{\theta} \max_{\phi} \{ \mathbb{E}[\log D_\phi(X)] + \mathbb{E}[\log(1 - D_\phi(G_\theta(Z)))] \}.$$

In the code we use the standard non-saturating generator loss, $-\mathbb{E}[\log D_\phi(G_\theta(Z))]$, because it provides a stronger gradient when the discriminator easily recognises early generated samples as fake.

A finite ReLU network driven by bounded latent input has bounded output, so a standard GAN cannot reproduce an unbounded heavy tail exactly. EV-GAN instead learns a bounded *tail-index function*

$$f^{\text{TIF}}(u) = -\frac{\log q(u)}{\log\{(1-u^2)/2\}}, \quad f^{\text{TIF}}(u) \rightarrow \gamma \quad (u \rightarrow 1),$$

and maps it back to the data scale using $H_u^{-1}(x) = \{(1-u^2)/2\}^{-x}$. The notebook adds the corrected-TIF terms used to improve approximation near the tail boundary.

ExceedGAN targets the conditional distribution above a high threshold. In one dimension, if $Y = X \mid X > q_X(1 - \delta)$, then

$$q_Y(1 - z) = q_X(1 - \delta z).$$

Thus generating increasingly extreme exceedances is an extrapolation problem. Fixed-level ExceedGAN anchors the generator at an empirical high threshold and uses transformed inputs $-\log Z$ and $-\log \delta$ with eLU hidden units motivated by the second-order extreme-value expansion.

Assessment should focus on the generated distribution rather than GAN losses alone. In the practical we compare histograms/scatter plots, upper-tail log-quantile curves and errors, acceptance rates into the upper orthant, and Kendall's τ as a simple bivariate dependence diagnostic.

Exercises

Exercise 1: Build and train a standard GAN

Open `01_standard_GAN.ipynb`. The example uses a one-dimensional bimodal target and $Z \sim N(0, 1)$.

1. Identify X , Z , G_θ , D_ϕ , d , and d' in the code, then run the forward-pass checks.
2. Train the GAN and inspect the discriminator and generator loss curves.
3. Compare the histogram of X with generated $\tilde{X} = G_\theta(Z)$ and inspect the learned map $z \mapsto G_\theta(z)$.
4. If time permits, change the hidden width J or replace the Gaussian latent distribution by a uniform distribution. What changes, and what does not?

Exercise 2: EV-GAN and heavy tails

Open `02_evGAN_vs_standard_GAN_1d.ipynb`. The target is a Burr distribution with $\gamma = 0.5$ and $\rho = -1$, and both models use $Z \sim U(0, 1)$.

1. Plot the Burr quantile function and its TIF. Verify visually that the quantile diverges while the TIF approaches γ .
2. Train the standard GAN and EV-GAN using the same adversarial objective.
3. Compare the fitted bulk distributions and the upper quantiles at 0.99, 0.995 and 0.999. Which architecture continues to extrapolate as $u \rightarrow 1$?
4. Plot the learned EV-GAN TIF against the true Burr TIF. As an extension, set `gamma = 0.9` and repeat the comparison.

Exercise 3: ExceedGAN for bivariate extremes

Open `03_exceedGAN_vs_evGAN_bivariate.ipynb`. The default experiment uses a Gumbel copula with $\mu = 2$, Burr margins with $\gamma = 0.5$ and $(\rho_1, \rho_2) = (-1, -3)$, and $\delta_n = (0.1, 0.1)^\top$.

1. Simulate the full sample, construct the upper-orthant training set $\mathcal{Q}(\delta_n)$, and inspect the thresholded log-log scatter plot.
2. Train the standard GAN, EV-GAN, and fixed-level ExceedGAN on the same exceedance data.
3. Compare generated samples using log-log scatter plots, marginal log-quantile error, and Kendall's τ .
4. Extension: set $\delta_n = (0.05, 0.05)^\top$, or change the Gumbel parameter to $\mu = 1.1$ or 10. How do tail accuracy and dependence change?

Questions for Discussion

- Why can a standard ReLU generator with bounded latent input reproduce the centre of a distribution while systematically missing its most extreme values?
- Which diagnostics would make you trust a generator for risk estimation: bulk fit, extreme quantiles, tail index, dependence, or some combination?
- What is gained by modelling the exceedance distribution directly with ExceedGAN rather than fitting the full distribution and then rejecting non-extreme generated samples?
- How would you adapt the comparison for higher-dimensional data, where pairwise plots and a single Kendall's τ are no longer sufficient?

References

- Goodfellow, I. et al. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems.
- Allouche, M., Girard, S. and Gobet, E. (2022). *EV-GAN: Simulation of extreme events with ReLU neural networks*. Journal of Machine Learning Research, 23(150), 1–39.
- Allouche, M., Girard, S. and Gobet, E. (2026). *ExceedGAN: Simulation above extreme thresholds using Generative Adversarial Networks*. Extremes.
- Edinburgh Summer School on Generative AI for Extremes: <https://gamex-network.github.io/school/>
- Special issue on Extremes & AI: <https://edin.ac/4akncn7>
- Original EV-GAN and ExceedGAN code: <https://github.com/michael-allouche/extreme-value-GAN>.